Université de Rouen Normandie

Master 1 – Génie Informatique et Logiciel

Projet Gestion de Flotte de véhicules

Rapport de Semaine 4

Microservices Conducteurs & Maintenance

Étudiants :
Ahcene Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Année universitaire :
2025 – 2026

Semaine :
4 / 7

4 avril 2026

Table des matières

1 Introduction et objectifs de la semaine **3**
1.1 Objectifs fixés . . . 3

2 Architecture de Persistance **4**
2.1 Isolation des instances . . . 4
2.2 Initialisation via Kubernetes . . . 4

3 Microservice Driver **5**
3.1 Entité Driver et relation avec DriverLicense . . . 5
3.2 Logique métier et conformité : LicenseScheduler . . . 6
3.3 Endpoints REST du service Driver . . . 6

4 Microservice Maintenance **7**
4.1 Entité MaintenanceRecord . . . 7
4.2 Logique Métier et Alertes . . . 8
4.2.1 Détection des retards . . . 8
4.2.2 Cloture avec calcul des prochaines échéances . . . 8
4.3 Endpoints REST du service Maintenance . . . 8

5 Tests de l'API REST et GraphQL avec Postman **9**
5.1 Authentification Keycloak . . . 9
5.2 Scénarios couverts et exemples de corps JSON . . . 9
5.3 Bilan des tests REST . . . 10
5.4 Requêtes GraphQL testées via Postman . . . 10
5.5 Bilan des tests GraphQL . . . 11

6 Exposition via le Gateway GraphQL **12**
6.1 Architecture du Gateway (Apollo Server) . . . 12
6.2 Résolveurs opérationnels . . . 12
6.3 Agrégation multi-services . . . 12

7 Architecture événementielle (Saga Pattern) **13**
7.1 Format d'enveloppe unifié (KafkaEventEnvelope) . . . 13
7.2 Transactions Compensatoires (Cycle de Maintenance) . . . 13
7.3 Topics Kafka et flux d'événements . . . 13
7.3.1 Topic : flotte.maintenance.events . . . 13
7.3.2 Topic : flotte.conducteurs.events . . . 14
7.3.3 Topic : flotte.assignments.events (flux Véhicule → Driver) . . . 14
7.4 Flux de données . . . 14
7.5 Remarque sur l'Outbox Pattern . . . 14

8 Tests automatisés et couverture de code **15**
8.1 Stratégie de tests 15
8.2 Mesure JaCoCo : profil Maven `unit-coverage` 15
8.3 Rapports JaCoCo captures (les trois services) 16
8.4 Synthèse des résultats (tests unitaires, métrique **lignes**) 17

1. Introduction et objectifs de la semaine

La semaine 4 s'inscrit après la mise en place de l'infrastructure (semaine 2) et le microservice **Véhicules** (semaine 3). Elle vise à completer la plateforme avec les services **Conducteurs** et **Maintenance**, la communication événementielle et la passerelle API, conformément au cahier des charges de la semaine.

1.1. Objectifs fixés

Les objectifs pédagogiques et fonctionnels de la semaine 4 sont les suivants :

— **Service Conducteurs :** exposition d'un CRUD complet, **validation et suivi des permis** (unicité, dates, alertes d'expiration), et **assignations** véhicule–conducteur cohérentes avec le service Véhicules (y compris via événements).

— **Service Maintenance : planification** des interventions, **historique** par véhicule, et **alertes préventives** (échéances, retard, seuils kilométriques).

— **Événements Kafka inter-services :** orchestration légère des transitions d'état et compensations selon un **pattern Saga** (chorégraphie).

— **API Gateway : agrégation GraphQL** multi-services (Véhicules, Conducteurs, Maintenance) derrière une API unifiée.

— **Authentification : intégration de Keycloak** (JWT, rôles) pour sécuriser les API REST et le Gateway.

> **Livrables attendus : trois services communicants** (Véhicules, Conducteurs, Maintenance) et une **API Gateway opérationnelle** (GraphQL), avec authentification Keycloak et échanges asynchrones Kafka entre services.

2. Architecture de Persistance

> **Principe appliqué :** Le pattern **Database per Service** est respecté strictement. Chaque microservice métier dispose de sa propre instance PostgreSQL (StatefulSet Kubernetes), garantissant une isolation totale des schémas et des données.

2.1. Isolation des instances

L'architecture de persistance suit le principe **une base de données par service métier** : chaque microservice dispose de sa propre instance PostgreSQL (StatefulSet) et de son propre nom de base.

- **Véhicules** : `postgres-vehicle-service` / `vehicle_db`.
- **Maintenance** : `postgres-maintenance-service` / `maintenance_db`.
- **Conducteurs** : `postgres-driver-service` / `driver_db`.

Les services **Événements** et **Localisation** ne sont pas traités dans ce livrable ; leur persistance dédiée sera ajoutée lorsque ces microservices seront développés.

2.2. Initialisation via Kubernetes

Les scripts SQL sont fournis via des **ConfigMaps** distincts, montés dans le bon pod PostgreSQL au `initdb`. La convention est la **même pour chaque service** : `01-<domaine>.sql` pour le schéma (DDL), `02-<domaine>-seed.sql` pour les données de test ; l'ordre lexicographique garantit le DDL avant les `INSERT`.

- `vehicle-db-init` : `01-vehicle.sql`, `02-vehicle-seed.sql`.
- `maintenance-db-init` : `01-maintenance.sql`, `02-maintenance-seed.sql`.
- `driver-db-init` : `01-driver.sql`, `02-driver-seed.sql`.

3. Microservice Driver

> **Objectif :** Gérer le cycle de vie complet des conducteurs : création de profils, suivi des permis de conduire et de leur validité, et coordination avec le service Véhicules lors des assignations.

Le service Driver est implémenté en Java / Spring Boot et déployé sur le port standard **8080**.

3.1. Entité Driver et relation avec DriverLicense

Le modèle de données repose sur une relation **Un-à-Plusieurs** entre l'entité `Driver` et `DriverLicense`. Le service implémente le *Soft Delete* via les annotations Hibernate `@SQLDelete` et `@SQLRestriction`, permettant de conserver l'historique tout en masquant les conducteurs désactivés des requêtes courantes.

```java
@Entity
@Table(name = "drivers")
@SQLDelete(sql = "UPDATE drivers SET deleted_at = CURRENT_TIMESTAMP
    WHERE id = ?")
@SQLRestriction("deleted_at IS NULL")
public class Driver {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    private String firstName;
    private String lastName;
    private String email;
    private String phone;
    private String employeeId;

    @Enumerated(EnumType.STRING)
    private DriverStatus status = DriverStatus.ACTIVE;

    @OneToMany(mappedBy = "driver", cascade = CascadeType.ALL)
    private List<DriverLicense> licenses = new ArrayList<>();

    private OffsetDateTime deletedAt;
}
```

Listing 1 – Entité Driver avec soft delete Hibernate

L'entité `DriverLicense` stocke les informations du permis (numéro, catégorie, date d'émission, date d'expiration, pays) et est liée au conducteur par une clé étrangère. Le champ `country` a été étendu à 50 caractères (initialement 2) pour accommoder les noms complets de pays.

3.2. Logique métier et conformité : LicenseScheduler

Le service Driver intègre une logique de surveillance proactive de la validité des permis. Une tâche planifiée (`LicenseScheduler`) s'exécute chaque nuit à minuit pour détecter les situations à risque avant qu'elles ne bloquent une opération.

```java
@Scheduled(cron = "0 0 0 * * *")
@Transactional(readOnly = true)
public void checkLicenseExpirations() {
    // Détection des permis expirant dans les 30 prochains jours
    LocalDate in30Days = LocalDate.now().plusDays(30);
    licenseRepository.findByExpiryDate(in30Days).forEach(license -> {
        eventProducer.sendLicenseExpiringEvent(license);
    });

    // Détection des permis expirés aujourd'hui
    LocalDate today = LocalDate.now();
    licenseRepository.findByExpiryDate(today).forEach(license -> {
        eventProducer.sendLicenseExpiredEvent(license);
    });
}
```

Listing 2 – Vérification proactive des permis (LicenseScheduler)

Les événements générés (`LICENSE_EXPIRING`, `LICENSE_EXPIRED`) sont publiés sur le topic Kafka `flotte.conducteurs.events` (constant `DriverEventProducer`) pour notifier les gestionnaires de flotte en temps réel.

3.3. Endpoints REST du service Driver

Le service expose une API REST complète en `snake_case` accessible via l'Ingress Kubernetes sur le préfixe `/api/drivers` :

Méthode	Endpoint	Description
GET	`/api/drivers`	Liste des conducteurs actifs (filtrable par statut).
POST	`/api/drivers`	Création d'un nouveau profil conducteur.
GET	`/api/drivers/{id}`	Détails d'un conducteur par UUID.
PUT	`/api/drivers/{id}`	Mise à jour complète du profil.
PATCH	`/api/drivers/{id}/status`	Mise à jour du statut (`active`, `on_tour`, etc.).
POST	`/api/drivers/{id}/licenses`	Ajout d'un permis de conduire au profil.
GET	`/api/drivers/{id}/licenses`	Liste des permis d'un conducteur.
DELETE	`/api/drivers/{id}`	Suppression logique du conducteur (Soft Delete).

Table 1 – Endpoints REST du service Driver

4. Microservice Maintenance

Objectif : Assurer la traçabilité complète des interventions techniques sur les véhicules de la flotte, de la planification initiale à la clôture, avec détection automatique des retards.

4.1. Entité MaintenanceRecord

L'entité centrale `MaintenanceRecord` modélise le cycle de vie d'une intervention technique. Elle utilise plusieurs énumérations (`MaintenanceType`, `MaintenanceStatus`, `MaintenancePriority`) définies en `SCREAMING_SNAKE_CASE` et alignées sur les types ENUM natifs PostgreSQL.

```java
@Entity
@Table(name = "maintenance_records")
public class MaintenanceRecord {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    private UUID vehicleId; // Lien logique (pas de FK inter-services)

    @Enumerated(EnumType.STRING)
    private MaintenanceType type; // PREVENTIVE, CORRECTIVE, INSPECTION

    @Enumerated(EnumType.STRING)
    private MaintenanceStatus status = MaintenanceStatus.SCHEDULED;

    @Enumerated(EnumType.STRING)
    private MaintenancePriority priority; // LOW, MEDIUM, HIGH, CRITICAL

    private LocalDate scheduledDate;
    private String description;
    private Double costEur;
    private Integer mileageAtService;
    private Integer nextServiceKm; // Seuil pour les alertes préventives
}
```

Listing 3 – Entité MaintenanceRecord

Le lien vers le service Véhicules est maintenu via un simple UUID (`vehicleId`) sans clé étrangère inter-bases, conformément aux principes de l'architecture microservices (pas de jointures cross-services).

4.2. Logique Métier et Alertes

4.2.1. Détection des retards

Une tâche planifiée (`@Scheduled`) s'exécute quotidiennement à 01h00 pour identifier les maintenances dont la date est dépassée et passer leur statut à `OVERDUE`.

```
@Scheduled(cron = "0 0 1 * * ?")
@Transactional
public void checkOverdueMaintenance() {
    List<MaintenanceRecord> scheduledRecords =
        repository.findByStatus(MaintenanceStatus.SCHEDULED);
    LocalDate today = LocalDate.now();
    for (MaintenanceRecord record : scheduledRecords) {
        if (record.getScheduledDate().isBefore(today)) {
            record.setStatus(MaintenanceStatus.OVERDUE);
            repository.save(record);
            eventProducer.publishAlert(AlertEvent.overdue(
                record.getVehicleId(), "Maintenance en retard"));
        }
    }
}
```

Listing 4 – Tâche de vérification des retards

4.2.2. Cloture avec calcul des prochaines échéances

L'endpoint `PATCH /api/maintenance/{id}/status` permet de clôturer une intervention en renseignant le cout, les notes techniques, le kilométrage au moment de la prestation et le kilométrage de la prochaine échéance. Ces données alimentent les alertes préventives futures.

4.3. Endpoints REST du service Maintenance

Méthode	Endpoint	Description
GET	`/api/maintenance`	Liste globale, filtrable par véhicule, statut ou priorité.
POST	`/api/maintenance`	Planification d'une nouvelle fiche d'intervention.
GET	`/api/maintenance/vehicle/{id}`	Historique complet d'un véhicule.
PUT	`/api/maintenance/{id}`	Mise à jour générale de la fiche (description, priorité, date, statut).
PATCH	`/api/maintenance/{id}/status`	Cloture technique (cout, kilométrage, notes, prochaine échéance).

Table 2 – Endpoints REST du service Maintenance

5. Tests de l'API REST et GraphQL avec Postman

> **Objectif :** Valider via Postman les API REST (services Driver et Maintenance, Ingress `http://flotte.local`) et le Gateway GraphQL (`/graphql`), avec le même jeton Keycloak pour l'ensemble des scénarios.

L'ensemble des tests est organisé dans une collection Postman couvrant trois périmètres : les services REST (Véhicules, Maintenance, Driver) et les requêtes GraphQL. Tous les appels transitent par l'Ingress NGINX du cluster Kubernetes, ce qui valide à la fois la logique applicative et la configuration du routage.

5.1. Authentification Keycloak

Avant d'exécuter les requêtes protégées, un token JWT est récupéré auprès du serveur Keycloak via le flux *Client Credentials* :

```
POST http://flotte.local/auth/realms/gestion-flotte/protocol/openid-
    connect/token

Body (x-www-form-urlencoded):
  grant_type    = client_credentials
  client_id     = postman-test
  client_secret = bxBExAzvhcd0U8VI9cQSmxl1fpzNtQxD
```

Listing 5 – Récupération du token Keycloak (client_credentials)

Le token retourné est ensuite injecté automatiquement dans le header `Authorization: Bearer <token>` de toutes les requêtes suivantes.

5.2. Scénarios couverts et exemples de corps JSON

Nous ne détaillons pas un scénario par endpoint : la collection Postman regroupe les requêtes nommées par route ; le tableau 3 synthétise les codes HTTP obtenus. Ci-dessous, quelques corps JSON représentatifs (les autres variantes — `PUT`, filtres `GET`, etc. — suivent la même structure que dans la collection).

Driver. Ont été validés : création et lecture de profil, `PUT` complet, `PATCH` de statut (dont `ON_TOUR`), ajout et liste de permis, `DELETE` logique (Soft Delete Hibernate). Le listing 6 montre un `POST` / `/api/drivers` minimal.

```
{
    "keycloak_user_id": "777a8400-e29b-41d4-a716-446655447777",
    "first_name": "Marc",
    "last_name": "Durand",
    "email": "marc.durand@test.com",
    "phone": "+33799999999",
    "employee_id": "EMP-999"
}
```

Listing 6 – Exemple de corps — POST `/api/drivers`

Maintenance. Ont été validés : création de fiche, listes et historique par véhicule, `PUT` puis clôture `PATCH` (coût, kilométrage, événement Kafka `MAINTENANCE_COMPLETED`).

Le listing 7 regroupe un `POST` de planification et un `PATCH` de clôture.

```
// POST : planification
{ "vehicle_id": "39e75ab3-7878-48a4-814e-f20c63fc99b7",
  "type": "CORRECTIVE", "priority": "HIGH",
  "scheduled_date": "2026-05-01", "description": "Changement des
   pneus" }

// PATCH : cloture (couts, notes, km)
{ "status": "COMPLETED", "cost_eur": 250.50,
  "notes": "Tout est OK, plaquettes changees.",
  "mileage_at_service": 45000,   "next_service_km": 60000 }
```

Listing 7 – Exemples — POST `/api/maintenance` et PATCH `/api/maintenance/{id}/status`

5.3. Bilan des tests REST

Service	Endpoint	Code attendu	Résultat
Driver	POST /api/drivers	201 Created	**OK**
Driver	GET /api/drivers	200 OK	**OK**
Driver	GET /api/drivers/{id}	200 OK	**OK**
Driver	PUT /api/drivers/{id}	200 OK	**OK**
Driver	PATCH /api/drivers/{id}/status	200 OK	**OK**
Driver	POST /api/drivers/{id}/licenses	201 Created	**OK**
Driver	GET /api/drivers/{id}/licenses	200 OK	**OK**
Driver	DELETE /api/drivers/{id}	204 No Content	**OK**
Maintenance	POST /api/maintenance	201 Created	**OK**
Maintenance	GET /api/maintenance	200 OK	**OK**
Maintenance	GET /api/maintenance/vehicle/{id}	200 OK	**OK**
Maintenance	PUT /api/maintenance/{id}	200 OK	**OK**
Maintenance	PATCH /api/maintenance/{id}/status	200 OK	**OK**

Table 3 – Bilan des 13 endpoints testés — Services Driver et Maintenance

L'ensemble des 13 endpoints testés retournent les codes HTTP attendus. Le Soft Delete Hibernate est correctement implémenté sur le service Driver. La clôture d'une intervention de maintenance déclenche bien la mise à jour du statut véhicule via Kafka, validant l'intégration inter-services.

5.4. Requêtes GraphQL testées via Postman

Les tests manuels reprennent le même jeton Keycloak que pour les API REST : requêtes `POST` vers `/graphql` sur l'Ingress (`http://flotte.local`), corps JSON au format GraphQL. Ont été validées des **queries de lecture** sur les trois domaines : liste des véhicules (`vehicles`), fiches de maintenance paginées (`maintenanceRecords` avec `items`), liste des conducteurs (`drivers` avec `items`), avec les champs utiles (identifiants, statuts, métadonnées). Les réponses renvoient un JSON conforme au schéma, sans erreur dans le tableau `errors` pour ces scénarios.

5.5. Bilan des tests GraphQL

Domaine	Opération testée	Réponse HTTP	Résultat
Véhicules	Query `vehicles` (liste, champs principaux)	200 OK	**OK**
Maintenance	Query `maintenanceRecords` (`items`, filtres de base)	200 OK	**OK**
Conducteurs	Query `drivers` (`items`, profils)	200 OK	**OK**

Table 4 – Bilan des requêtes GraphQL validées via Postman (Gateway, JWT Ingress).

Ces trois requêtes couvrent les resolveurs racine exposés par le Gateway ; les enchaînements d'agrégation (relations entre types) sont présentés section 6.3.

6. Exposition via le Gateway GraphQL

> **Objectif :** Remplacer les stubs de la semaine 3 par les résolveurs opérationnels des services Driver et Maintenance, et activer l'agrégation multi-services.

6.1. Architecture du Gateway (Apollo Server)

Le Gateway GraphQL est une application Node.js dédiée, buildée dans une image Docker `graphql-gateway:latest` et déployée sur le port 4000 du cluster Kubernetes. Les requêtes sont résolues via un `Context` instanciant des clients REST (Axios) qui injectent systématiquement le jeton d'authentification reçu :

- **VehicleClient** : requêtes vers `http://vehicle-service:8080`
- **DriverClient** : requêtes vers `http://driver-service:8080`
- **MaintenanceClient** : requêtes vers `http://maintenance-service:8080`

La configuration de l'**Ingress** route le trafic de `/graphql` vers cette passerelle. Une correction du header `Authorization` a été apportée pour que le JWT soit systématiquement transmis aux services backend.

6.2. Résolveurs opérationnels

Les trois familles de resolveurs sont désormais pleinement fonctionnelles. Le fichier `index.ts` assemble les resolveurs par spread operator : les nouvelles entrées écrasent les stubs correspondants sans modifier la structure d'agrégation. Les tests Postman sur les queries racine et le tableau de résultats sont décrits section 5.4.

6.3. Agrégation multi-services

L'API Gateway permet d'agréger les données de façon transparente pour le client. Les relations suivantes sont désormais résolubles en une seule requête GraphQL :

- **Véhicule** ↔ **Maintenance** : Affichage de l'historique technique depuis la fiche véhicule.
- **Véhicule** ↔ **Conducteur** : Détail de l'assignation courante d'un véhicule vers le profil du conducteur (et ses permis).
- **Maintenance** ↔ **Conducteur** : Résolution du technicien assigné à une intervention.

7. Architecture événementielle (Saga Pattern)

> **Principe :** Le pattern Saga par **Chorégraphie** assure la cohérence éventuelle entre les services sans orchestrateur central. Chaque service réagit aux événements publiés par les autres et publie à son tour ses propres événements de résultat.

7.1. Format d'enveloppe unifié (KafkaEventEnvelope)

Tous les événements inter-services partagent un format d'enveloppe standardisé, facilitant le traçage et l'audit :

```
{
  "eventId": "625c87b9-...",
  "eventType": "MAINTENANCE_COMPLETED",
  "timestamp": 1775220800.293,
  "payload": {
    "record_id": "7e3638d7-...",
    "vehicle_id": "39e75ab3-...",
    "cost_eur": 250.50
  },
  "metadata": { "triggered_by": "maintenance-service" }
}
```

Listing 8 – Format KafkaEventEnvelope unifié

7.2. Transactions Compensatoires (Cycle de Maintenance)

Pour garantir l'intégrité des données lors d'une transition d'état impliquant plusieurs services, un mécanisme de compensation a été mis en œuvre :

1. **T1 (Action)** : Le service Maintenance crée une fiche (`MAINTENANCE_SCHEDULED`) ; lors du passage en `IN_PROGRESS`, il publie `MAINTENANCE_STARTED` sur `flotte.maintenance.event`
2. **T2 (Réaction)** : Le service Véhicule consomme `MAINTENANCE_STARTED` et tente de passer le véhicule en `in_maintenance`.
3. **C1 (Compensation)** : En cas d'échec, le service Véhicule publie `MAINTENANCE_REJECTED`. Le service Maintenance consomme cet événement et appelle `cancelRecord` (statut `CANCELLED`, note de traçabilité).

7.3. Topics Kafka et flux d'événements

7.3.1. Topic : flotte.maintenance.events

Le service Véhicules consomme ces messages pour ajuster l'état opérationnel du parc :

— `MAINTENANCE_STARTED` → Statut véhicule : `in_maintenance`.

— `MAINTENANCE_COMPLETED` / `MAINTENANCE_CANCELLED` → Statut véhicule : `available`.

7.3.2. Topic : flotte.conducteurs.events

Ce topic centralise les événements du cycle de vie des conducteurs :

— `DRIVER_CREATED` / `DRIVER_UPDATED` : Notifient les autres services des changements de profil.

— `LICENSE_EXPIRING` / `LICENSE_EXPIRED` : Alertes proactives générées par le `LicenseScheduler`.

7.3.3. Topic : flotte.assignments.events (flux Véhicule → Driver)

Le service Driver écoute ce topic : sur `VEHICLE_ASSIGNED`, le conducteur passe en `ON_TOUR` ; sur `VEHICLE_UNASSIGNED`, il repasse en `ACTIVE`. Ces événements sont publiés par le service Véhicules lors des assignations / désassignations, sans appel REST direct entre les deux services.

7.4. Flux de données

L'architecture repose sur deux modes de communication complémentaires :

— **Flux synchrones (Gateway GraphQL)** : Les requêtes de lecture transitent par le Gateway qui appelle en parallèle les services REST concernés et assemble les réponses JSON en un objet unifié.

— **Flux asynchrones (Kafka)** : Les changements d'état critiques (cycle de vie Maintenance, mise à jour kilométrique, assignation conducteur) utilisent le bus de messages pour garantir le découplage et la cohérence éventuelle.

7.5. Remarque sur l'Outbox Pattern

Une réflexion a été conduite sur la fiabilité de la double écriture (base de données + Kafka). L'**Outbox Pattern** constitue le standard industriel pour garantir l'atomicité parfaite entre persistance et publication d'événements. Toutefois, compte tenu de la complexité opérationnelle qu'implique un outil comme *Debezium*, nous avons opté pour le **Direct Publishing** au sein de transactions Spring `@Transactional`. Cette approche, combinée aux transactions compensatoires du Saga, offre un excellent compromis robustesse/maintenabilité pour notre périmètre.

8. Tests automatisés et couverture de code

Objectif : Garantir la robustesse via une couverture minimale de **80% en lignes de code** sur les tests **unitaires** des trois services métier (**vehicle-service**, **maintenance-service**, **driver-service**), mesurée avec JaCoCo.

8.1. Stratégie de tests

L'approche de test couvre le périmètre métier via deux niveaux complémentaires :

— **Tests unitaires (JUnit 5 + Mockito, `@WebMvcTest` pour les contrôleurs)** : validation des services, producteurs/consommateurs Kafka mockés, et endpoints REST sans lever tout le contexte Spring sauf pour les tranches nécessaires.

— **Tests d'intégration (`@SpringBootTest`, MockMvc, base H2 en profil `test`)** : chargement du contexte applicatif, vérification des flux HTTP et de la persistance sur une base embarquée.

8.2. Mesure JaCoCo : profil Maven `unit-coverage`

La couverture demandée pour le cours porte sur les **tests unitaires** uniquement. Nous avons donc ajouté un profil Maven `unit-coverage` dans chaque service : Surefire **exclut** les classes `*IntegrationTest`, le dossier `intégration/` et les tests de chargement de contexte `*ApplicationTests`, afin que JaCoCo ne comptabilise pas ces exécutions dans le rapport "unitaire".

Génération du rapport HTML (à exécuter dans le répertoire du service concerné, par ex. `services/driver-service/`) :

```
./mvnw clean test -Punit-coverage
```

Listing 9 – Commandes Maven (Wrapper) pour la couverture unitaire

Le rapport est alors produit sous `target/site/jacoco/index.html` (un rapport par module). Chaque exécution **régénère** ce dossier : pour conserver la mesure "uniquement unitaire", il faut lancer la commande ci-dessus avec le profil `unit-coverage` avant de capturer les captures d'écran.

Vérification du seuil à 80% (lignes) en phase `verify` (optionnel, fait échouer la build si le seuil n'est pas atteint) :

```
./mvnw clean verify -Punit-coverage -Djacoco.skip.unit.check=false
```

Listing 10 – Vérification JaCoCo avec seuil minimal

Par défaut, le `jacoco:check` du profil est **désactivé** (`jacoco.skip.unit.check=true`) pour ne pas bloquer le développement ; passer `jacoco.skip.unit.check=false` active la contrainte.

8.3. Rapports JaCoCo captures (les trois services)

Les figures 1 à 3 présentent la vue d'ensemble JaCoCo pour chaque service, obtenue après `./mvnw clean test -Punit-coverage`. Les fichiers sources des captures sont stockés dans le répertoire `docs/rapport-S4/image/` (`jacoco-driver.png`, `jacoco-maintenance.png`, `jacoco-vehicle.png`).

driver-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.flotte.driver.config		0%		0%	12	12	22	22	10	10	1	1
com.flotte.driver.controllers		55%		n/a	4	9	4	12	4	9	0	1
com.flotte.driver.models		88%		n/a	4	40	6	54	4	40	0	2
com.flotte.driver		37%		n/a	1	2	2	3	1	2	0	1
com.flotte.driver.events		98%		n/a	1	13	1	23	1	13	0	7
com.flotte.driver.services		100%		75%	4	23	0	103	0	15	0	1
com.flotte.driver.events.producers		100%		n/a	0	5	0	37	0	5	0	1
com.flotte.driver.dto		100%		n/a	0	6	0	6	0	6	0	6
com.flotte.driver.events.consumers		100%		100%	0	6	0	22	0	4	0	2
com.flotte.driver.models.enums		100%		n/a	0	2	0	13	0	2	0	2
com.flotte.driver.jobs		100%		n/a	0	5	0	17	0	5	0	1
Total	151 of 1,392	89%	8 of 24	66%	26	123	35	312	20	111	1	25

FIGURE 1 – Rapport JaCoCo (vue globale) — `driver-service`, couverture sur tests unitaires (`-Punit-coverage`).

maintenance-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.flotte.maintenance.config		0%		0%	11	11	19	19	9	9	1	1
com.flotte.maintenance.service		91%		62%	23	47	4	83	1	18	0	1
com.flotte.maintenance.events		93%		60%	7	30	3	56	3	25	1	7
com.flotte.maintenance.model		90%		n/a	5	36	5	54	5	36	0	4
com.flotte.maintenance.controller		85%		n/a	1	7	1	9	1	7	0	1
com.flotte.maintenance		37%		n/a	1	2	2	3	1	2	0	1
com.flotte.vehicle.events		100%		n/a	0	4	0	4	0	4	0	4
com.flotte.maintenance.dto		100%		n/a	0	3	0	3	0	3	0	3
Total	177 of 1,215	85%	30 of 72	58%	48	140	34	231	20	104	2	22

FIGURE 2 – Rapport JaCoCo (vue globale) — `maintenance-service`, couverture sur tests unitaires (`-Punit-coverage`).

vehicle-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.flotte.vehicle.config		0%		0%	12	12	23	23	10	10	1	1
com.flotte.vehicle.services		88%		58%	12	26	2	127	7	20	0	1
com.flotte.vehicle.models		88%		n/a	5	45	5	52	5	45	0	2
com.flotte.vehicle.events.consumers		88%		58%	4	13	3	35	0	5	0	1
com.flotte.vehicle		37%		n/a	1	2	2	3	1	2	0	1
com.flotte.vehicle.events		98%		n/a	1	14	1	23	1	14	0	6
com.flotte.vehicle.dto		100%		n/a	0	7	0	15	0	7	0	6
com.flotte.vehicle.events.producers		100%		n/a	0	5	0	16	0	5	0	1
com.flotte.vehicle.controllers		100%		n/a	0	10	0	13	0	10	0	1
com.flotte.vehicle.models.enums		100%		n/a	0	2	0	10	0	2	0	2
com.flotte.maintenance.model		100%		n/a	0	1	0	6	0	1	0	1
com.flotte.maintenance.events		100%		n/a	0	1	0	1	0	1	0	1
Total	202 of 1,509	86%	14 of 28	50%	35	138	36	324	24	122	1	24

FIGURE 3 – Rapport JaCoCo (vue globale) — `vehicle-service`, couverture sur tests unitaires (`-Punit-coverage`).

8.4. Synthèse des résultats (tests unitaires, métrique lignes)

Après exécution de `./mvnw clean test -Punit-coverage` et analyse des rapports, les trois services dépassent le seuil de 80% en couverture de **lignes** sur le bundle (packages applicatifs). Les pourcentages exacts peuvent varier légèrement selon l'évolution du code ; les figures ci-dessus font foi au moment du rapport.

Service	Couverture lignes (ordre de grandeur)	Objectif
`driver-service`	$\approx 89\%$	$\geq 80\%$
`maintenance-service`	$\approx 85\%$	$\geq 80\%$
`vehicle-service`	$\approx 89\%$	$\geq 80\%$

Table 5 – Synthèse indicative (profil `unit-coverage`, métrique lignes JaCoCo).

La couverture de **branches** reste plus basse sur certains modules (consommateurs Kafka, chemins d'erreur), ce qui est classique ; l'exigence du module a été exprimée sur les **lignes** via le `jacoco:check` du profil.